A JURY PROJECT REPORT

Submitted in partial fulfillment of the requirements

for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE & ENGINEERING

SUBJECT: DATA WAREHOUSING AND DATA MINING (DWDM)

JURY II: UNITS III & IV

"Customer Behavioral Analysis and Loan Default Classification"

A Data Mining Approach to Credit Risk on the PKDD'99 Dataset

SUBMITTED BY:

Dhanush Pillay ADT24SOCB0765

Shubhangini Dixit ADT24SOCB1156

Semester: 5th Semester / TY BDCE

INTERNAL FACULTY GUIDE:

Dr. Reena Pagare

Department of Computer Science & Engineering

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

MIT ART, DESIGN AND TECHNOLOGY UNIVERSITY, PUNE

Loni Kalbhor, Pune, MH, India – 412201

Contents

1 Introduction **1**

1.1 Problem Definition 1

1.2 Motivation and Business Context 1

1.3 Project Objectives 2

1.4 Scope and Deliverables 2

1.5 Jury II Evaluation Coverage 3

1.6 Organization of the Report 3

2 Literature Review and Background **6**

2.1 The PKDD'99 Financial Dataset 6

2.2 Classification Algorithms 7

2.2.1 Decision Tree 7

2.2.2 Naive Bayes 7

2.2.3 Random Forest 7

2.2.4 Support Vector Machine 7

2.2.5 K-Nearest Neighbors 8

2.2.6 Gradient Boosting 8

2.3 Class Imbalance and SMOTE 8

2.4 Evaluation Metrics under Imbalance 8

2.5 Model Selection and Cross-Validation 9

2.6 Related Work and Positioning 9

3 System Architecture and Methodology **10**

3.1 Pipeline Overview 10

3.2 Technology Stack 11

3.3 Pipeline Stages 12

3.3.1 Data Assembly 12

3.3.2 Cleaning and Target Construction 12

3.3.3 Feature Engineering 12

3.3.4 Model Training and Evaluation 12

3.3.5 Artifact Generation and Serving 13

3.4 Input and Output Specification 13

3.5 Leakage Controls 14

3.6 Evaluation Protocol 14

3.7 Reproducibility 15

4 Data Preprocessing and Feature Engineering 17

4.1 Source Tables and Join Logic 17
4.2 Cleaning and Validation 17
4.3 Class Labels and Target Definition 19
4.4 Pre-Loan Behavioural Features 19
4.5 Loan Terms and Demographics 21
4.6 Final Feature Table 21
4.7 Transformation and Train/Test Split 22
4.8 Data Quality Observations 22

5 Exploratory Data Analysis and Visualizations 24

5.1 Class Distribution 24
5.2 Features Associated with Default 25
5.3 Balance Separation 26
5.4 Age and Gender 27
5.5 Collinearity and Redundancy 27
5.6 Loan Size Relative to Balance 28
5.7 Behavioural Segments 29
5.8 Summary of Findings 31

6 Classification Models and Performance Evaluation 32

6.1 Classifiers and Configurations 32
6.2 Training Pipeline 33
6.3 Hyperparameter Tuning 34
6.4 Performance Metrics and Held-Out Test Results 35
6.5 Validation Test: Repeated Cross-Validation 37
6.6 Ranking Curves 39
6.7 Feature Importance 40
6.8 Pattern Matching with Apriori and FP-Growth 41
6.9 Discussion 42
6.10 Interactive Review Console 43

7 Results, Conclusions and Future Scope 45

7.1 Summary of Results 45
7.2 Key Risk Signals 46

7.3 Innovation Added to the Classification Algorithm 46
7.4 Business Recommendations 47
7.5 Limitations 48
7.6 Future Scope 49
7.7 Conclusion 50

References **51**

List of Figures

3.1 End-to-end architecture of the loan default classification project. The pipeline is batch-oriented: each stage reads files from the previous stage and writes a versioned artifact. 10

5.1 Distribution of the binary target: 606 good loans and 76 defaults (11.14%). . . 24
5.2 Correlation of each numeric feature with the default label. Balance features are negative; loan size and payment are positive. 25
5.3 Box plots of selected behavioural features by class. The balance distributions separate more clearly than transaction frequency. 26
5.4 Age distribution by outcome. Defaults occur across all ages rather than in a single band. 27
5.5 Correlation matrix of the numeric features. The cash-flow pair is nearly duplicated, and loan amount tracks monthly payment. 28
5.6 Default rate by balance and loan amount. The low-balance, high-amount quadrant has the highest risk. 29
5.7 Loans in the two behavioural segments, plotted in the activity and balance space. 30
5.8 Default rate by segment. The quieter, lower-balance segment has the higher rate, but the difference is modest. 31

6.1 Hyperparameter sensitivity for the baseline arm on the training split. F1 is flat over a broad region and falls at the extremes. 35
6.2 Confusion matrices on the held-out test set with default hyperparameters. The two rows show the baseline and SMOTE arms. 37
6.3 Tuning gain per configuration under repeated cross-validation. The selected Gradient Boosting baseline is marked. 39
6.4 ROC curves by imbalance arm on the held-out test set, default hyperparameters. 39
6.5 Precision-recall curves with average precision on the held-out test set, default hyperparameters. The dashed baseline is the 11.14% default rate. 40
6.6 Feature importance from the tuned Random Forest. Balance features dominate; demographic features contribute little. 41

List of Tables

1.1 Deliverable coverage for Jury II 3
1.2 Coverage of the Jury II evaluation parameters. 4

2.1 PKDD'99 tables used in the project 6

3.1 Tools used in the project and their roles. 11
3.2 Inputs and outputs at each stage of the pipeline. 13

4.1 Data checks applied before feature construction. 18
4.2 Target definition and class balance. 19
4.3 The 13 pre-loan behavioural features. All values are computed from transactions dated strictly before the loan grant date. 20
4.4 The 18 features used by the classifiers, by group. 21
4.5 Stratified train/test split. 22

5.1 Default rate by pre-loan balance and loan amount, split at the respective medians. 29
5.2 Profiles of the two behavioural segments. The target was not used to form the clusters. 30

6.1 Classifiers evaluated in the project. 32
6.2 Hyperparameter search results, ranked by inner cross-validation F1. 34
6.3 Held-out test performance with default hyperparameters (171 loans, 19 defaults). 36
6.4 Repeated stratified five-fold cross-validation, ten repeats (50 estimates), ranked by tuned F1. The last two columns describe the tuned configuration. 37
6.5 Default-oriented association rules after redundancy pruning. Support is the share of the 682 accounts; coverage is the share of the 76 defaults. 42

7.1 The two strongest configurations under repeated cross-validation. 45

Chapter 1

Introduction

1.1 Problem Definition

Banking institutions sit on large volumes of operational data: account histories, transactions, demographics, and loan records. The information that predicts whether a borrower will repay a loan is usually already present in that history before the loan is granted, but it is buried in millions of individual transactions and is not directly usable by a credit officer.

This project addresses the resulting decision problem as a supervised binary classification task. Given a loan application and the applicant's pre-loan transaction history, the system estimates whether the loan will be repaid (a *good* loan) or will default (a *bad* loan). The task is performed on the PKDD'99 Financial Dataset, which contains 682 loans and roughly one million account transactions from a Czech bank between 1993 and 1998. Of the 682 loans, 606 were repaid and 76 defaulted, giving a base default rate of 11.14 percent.

The central constraint is temporal: every behavioral feature is computed only from transactions dated strictly before the loan grant date. Post-loan activity is never used, so the model reproduces what the bank could have known at decision time rather than what it learned afterwards.

1.2 Motivation and Business Context

Loan defaults are direct losses, while rejected good customers are opportunity costs. A model that separates the two groups well allows the bank to route applications into a review queue, ask for additional collateral, or adjust the offered amount. The cost of the two error types is not symmetric: approving a future defaulter is usually more expensive than reviewing a good customer twice.

The dataset is also strongly imbalanced. With only 11.14 percent positives, a trivial classifier that predicts "repaid" for every application reaches 88.86 percent accuracy while catching zero defaulters. Accuracy is therefore not a useful headline metric here, and the evaluation has to focus on the positive class: precision, recall, F1-score, Average Precision, and ROC-AUC. This project treats that methodological point as part of the problem, not as an afterthought.

1.3 Project Objectives

The project covers the complete data mining pipeline required by Units III and IV of the syllabus. Its objectives are:

1. To clean and integrate the raw PKDD'99 tables into a single loan-level analytical dataset.
2. To engineer behavioral features from pre-loan transactions only, and to verify that no post-loan information leaks into the feature set.
3. To perform exploratory data analysis and visualize the behavioral differences between repaid and defaulted loans.
4. To train and compare six classification algorithms: Decision Tree, Naive Bayes, Random Forest, Support Vector Machine (SVM), K-Nearest Neighbors (KNN), and Gradient Boosting.
5. To handle class imbalance explicitly by training each classifier twice, once on the raw data and once on a SMOTE-balanced training set.
6. To tune hyperparameters and select the final model using repeated stratified cross-validation rather than a single train–test split.
7. To interpret the selected model through feature importance and association rule mining, and to translate the findings into business recommendations.

1.4 Scope and Deliverables

The project scope is limited to binary loan-default classification on historical data. It does not include live scoring, credit-policy design, or causal analysis. The supplementary components are customer segmentation with KMeans, association rule mining with Apriori and FP-Growth, and a Streamlit decision-support console that reads the exported result files.

Table 1.1 maps each expected deliverable to the artifact that implements it.

Deliverable	Where it is implemented
Problem definition and objectives	This chapter and `docs/Problem_Statement.md`
Dataset description and source	Chapter 2; raw CSVs in `data/`
Data preprocessing pipeline	Chapter 4; `01_preprocessing.ipynb`, `02_features.ipynb`
EDA and visualizations	Chapter 5; `03_eda.ipynb`; 8 EDA figures, 14 project figures in total
Classification algorithms	Chapter 6; `04_modeling.ipynb`
Performance comparison	Chapter 6; `metrics_comparison.csv`, `tuning_results.csv`, `cv_comparison.csv`
Conclusions and recommendations	Chapter 7; `docs/Jury2_Report.md`
Association rule mining (Unit III)	Section 6.8; `tools/pattern_mining.py`
Project report and presentation	This document and the executed notebooks

Table 1.1: Deliverable coverage for Jury II

1.5 Jury II Evaluation Coverage

Table 1.2 maps the evaluation parameters for Jury II to the parts of this report that address them. The table is an index, not a substitute for the chapters: each row points to the analysis, table, or figure that contains the evidence.

1.6 Organization of the Report

Chapter 2 reviews the dataset, the classifiers, imbalance handling, and the evaluation metrics. Chapter 3 describes the system architecture, the input and output specification, leakage controls, and the evaluation protocol. Chapter 4 documents data cleaning and feature engineering. Chapter 5 presents the exploratory analysis. Chapter 6 reports the model implementation, tun-

Evaluation parameter	Coverage in this report
Dataset	Chapter 2, Table 2.1; source joins and cleaning in Chapter 4.
Architecture diagram	Figure 3.1 and the stage description in Section 3.3.
Input / output	Section 3.4, Table 3.2; the 18-feature model input and scored output are stated there.
Classes / labels	Section 4.3, Table 4.2; the A/C versus B/D mapping and the 606/76 balance.
Pattern matching (FP-Growth, Apriori)	Section 6.8, Table 6.5; both algorithms, the 41 itemsets, and the two default rules.
Comparison with other classification algorithms	Sections 6.4 and 6.5, Tables 6.3 and 6.4; six classifiers under baseline and SMOTE.
Classification algorithm	Chapter 2 and Sections 6.1 and 6.2; the six learners and their pipelines.
Innovation added to the classification algorithm	Section 7.3; the pre-loan representation, balance-capacity view, two-arm validation, and rule-plus-model review layer.
Performance metrics	Section 3.6 and Sections 6.4 and 6.6; accuracy, precision, recall, F1, AP, ROC-AUC, and confusion matrices.
Chart	Chapter 5 and Figures 6.4 and 6.5; the Streamlit console in Section 6.10 presents the same charts.

Table 1.2: Coverage of the Jury II evaluation parameters.

ing, comparison, feature importance, and pattern mining. Chapter 7 states the conclusions, innovation, limitations, and future scope. Section 1.5 maps each Jury II evaluation parameter to these chapters.

Chapter 2

Literature Review and Background

2.1 The PKDD'99 Financial Dataset

This project uses the **PKDD'99 Financial Dataset**, also known as the Berka Dataset, released for the Discovery Challenge at the 1999 European Conference on Principles and Practice of Knowledge Discovery in Databases. It contains anonymized real-world records from a Czech bank covering 1993 to 1998.

The raw data is highly normalized and spread across eight tables. Five are used here. Table 2.1 summarizes them.

Table	Rows	Role in this project
`loan.csv`	682	One row per granted loan: account, grant date, amount, duration, installment, and status code. Source of the target.
`trans.csv`	1,056,320	One row per account transaction, 1993-01-01 to 1998-12-31. Source of all behavioral features.
`client.csv`	5,369	Client identifier, birth number (encodes date of birth and gender), and district. Source of age and gender.
`account.csv`	4,500	Account identifier, district, statement frequency, and creation date. Used for provenance.
`disp.csv`	5,369	Links clients to accounts as OWNER or DISPONENT. Only OWNER links are used, so every loan maps to one client.

Table 2.1: PKDD'99 tables used in the project

The dataset is a standard benchmark for relational data mining because it combines long transaction histories with a small, well-defined loan outcome. This project reads the raw CSV extracts directly. No warehouse, BI tool, or external database is part of the pipeline.

2.2 Classification Algorithms

Six supervised learners are compared. They were chosen to cover different inductive biases rather than to maximize the leaderboard.

2.2.1 Decision Tree

A decision tree partitions the feature space with axis-aligned splits chosen to reduce impurity. It captures non-linear interactions, is easy to explain, and serves as an interpretable baseline. Its main weakness is variance: small changes in the training data can change the tree structure. The search in this project tunes maximum depth, minimum samples per leaf, and the splitting criterion.

2.2.2 Naive Bayes

Gaussian Naive Bayes models each feature as a normal distribution conditional on the class and assumes conditional independence. It is fast and needs very little data, but the independence assumption is clearly violated here because balance and volume features are correlated. It is included as a probabilistic baseline, not as a candidate for deployment.

2.2.3 Random Forest

Random Forest [9] averages many decorrelated decision trees, each grown on a bootstrap sample with a random subset of features at every split. The averaging reduces variance and the out-of-bag behaviour gives a stable impurity-based importance ranking. It is typically strong on small tabular problems and is the reference model for feature importance in this report.

2.2.4 Support Vector Machine

An SVM finds a maximum-margin separating hyperplane, using the kernel trick to represent non-linear boundaries. With 682 rows and 18 features it is computationally comfortable. Because the ROC and precision–recall curves need probability estimates, the SVM is wrapped in `CalibratedClassifierCV` with a sigmoid calibration and cross-validated calibration folds.

2.2.5 K-Nearest Neighbors

KNN classifies a point by the majority label among its k nearest training neighbours. It makes no global assumption about the decision boundary and provides a distance-based sanity check. It is sensitive to feature scaling, which is why the scaler is part of the pipeline, and it usually struggles when the positive class is rare.

2.2.6 Gradient Boosting

Gradient Boosting [10] fits shallow trees sequentially, each new tree correcting the residual errors of the current ensemble. With a low learning rate and subsampling it often outperforms a single tree and a random forest on structured tabular data. The search tunes the number of estimators, depth, learning rate, and subsample fraction.

2.3 Class Imbalance and SMOTE

The target is imbalanced at roughly 9:1. A classifier optimized for accuracy will usually predict the majority class and ignore defaulters. Two responses are common: cost-sensitive learning and resampling. This project uses SMOTE [8] as the resampling arm, because it is the standard oversampling method and is easy to place inside a scikit-learn pipeline.

SMOTE generates synthetic minority examples by interpolating between a minority point and its nearest minority neighbours. It increases the minority region rather than duplicating points, which reduces the risk of exact overfitting. The comparison in this report is deliberately two-armed: every classifier is trained once on the raw training data and once on a SMOTE-balanced copy, with all resampling fitted inside the training fold only. As Chapter 6 shows, the balanced arm does not help the stronger models, a result that only repeated validation made visible.

2.4 Evaluation Metrics under Imbalance

Accuracy is reported but never used for selection. The primary metrics are:

- **Precision:** of the loans flagged as default, the fraction that actually defaulted.
- **Recall:** of the loans that actually defaulted, the fraction that were flagged.

- **F1-score:** the harmonic mean of precision and recall, used as the selection criterion.
- **Average Precision (AP):** the area under the precision–recall curve, which summarizes ranking quality for the positive class.
- **ROC-AUC:** the probability that a random defaulter is ranked above a random repayer.

ROC-AUC can look flattering when the positive class is rare, because a large number of true negatives inflates the false-positive-rate denominator. Average Precision does not have that property, so both are reported. The comparison also uses a confusion matrix per configuration so that the trade-off between missed defaults and false alarms stays visible.

2.5 Model Selection and Cross-Validation

A single 75/25 split leaves about 19 defaults in the test set. At that size, the ranking of twelve configurations is dominated by split noise. The project therefore separates three questions: a readable held-out table at default hyperparameters, a hyperparameter search on the training split, and a repeated stratified cross-validation over all configurations. The final choice uses the repeated 5-fold $\times$ 10 protocol, which yields 50 estimates per configuration and makes the fold-to-fold spread explicit. This follows the standard CRISP-DM emphasis on evaluation as an iterative, evidence-based stage rather than a single number [11].

2.6 Related Work and Positioning

The Berka dataset has been used for loan-default prediction, customer segmentation, and relational feature construction. Much of the published work either uses post-loan information, which is unavailable at decision time, or reports a single train–test split on a small positive class. This project is positioned differently in three ways: the feature set is strictly pre-loan, the comparison includes a balanced and an unbalanced arm for every model, and model selection rests on repeated cross-validation with the uncertainty reported. The association rule mining step is included as a descriptive complement, not as a competitor to the classifiers.

Chapter 3

System Architecture and Methodology

This chapter describes how the project turns five raw PKDD'99 tables into a scored loan portfolio. The work is organised as an offline analytical pipeline with four layers: data assembly, feature engineering, modelling, and serving. Each layer writes a durable artifact, so a result shown in the report or the dashboard can be traced back to a file produced by an earlier stage.

3.1 Pipeline Overview

Figure 3.1 shows the end-to-end flow. The data layer reads the loan, transaction, client, account, and disposition tables. The feature layer builds one row per loan using only transactions dated before the loan was granted. The modelling layer trains six classifiers under two imbalance settings, tunes them, and evaluates them with a held-out split and repeated cross-validation. The serving layer exposes the resulting metrics, feature importances, risk scores, and mined patterns through a Streamlit console.

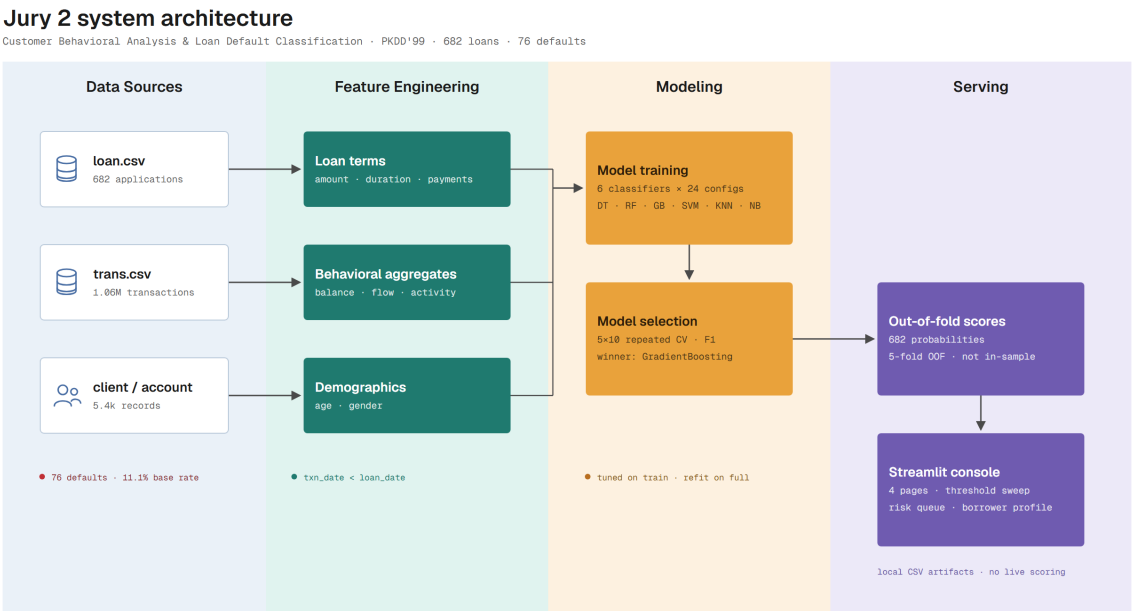


Figure 3.1: End-to-end architecture of the loan default classification project. The pipeline is batch-oriented: each stage reads files from the previous stage and writes a versioned artifact.

The design keeps the analytical work separate from the presentation work. Notebooks produce CSV and PNG artifacts under `outputs/`; the dashboard only reads those artifacts and does not retrain a model when a page is opened. This makes the reported numbers stable and allows the console to run on a machine without the full training environment.

3.2 Technology Stack

The implementation uses open-source Python tools. Table 3.1 lists the main components and their responsibilities.

Table 3.1: Tools used in the project and their roles.

Tool	**Role in the project**
Python 3.13.5	Main programming language for all notebooks and scripts.
pandas 2.2.3	Reading the CSV extracts, joining tables, and building loan-level features.
NumPy 2.2.4	Numeric operations and array handling inside pandas and scikit-learn.
scikit-learn 1.7.0	Classifiers, pipelines, scaling, cross-validation, and hyperparameter search.
imbalanced-learn 0.14.2	SMOTE oversampling inside the modelling pipeline.
matplotlib 3.10.3	Base plotting for the figures in this report.
seaborn 0.13.2	Distribution, box, and correlation plots.
mlxtend	Apriori and FP-Growth pattern mining.
Jupyter Notebook	Literate analysis; notebooks are executed in order with nbconvert.
Streamlit	Interactive review console for the exported results.

The versions shown for the core Python libraries are the ones recorded for the analysis environment. The notebooks also rely on the standard library modules `pathlib`, `json`, and `warnings`. No database server or distributed compute engine is required; the largest table, `trans.csv`, has 1,056,320 rows and fits comfortably in memory on a laptop.

3.3 Pipeline Stages

3.3.1 Data Assembly

The five source files are loaded with explicit column types where needed. The disposition table links clients to accounts. A client can be an owner or a disbursent, and the same account can have more than one client. Only rows with `disp.type = OWNER` are used, because the loan is associated with the account owner rather than with every person who can operate the account. The account table supplies the district, the client table supplies the birth number, and the loan table supplies the target and the loan terms.

3.3.2 Cleaning and Target Construction

Dates in the source files use the `%y%m%d` format and are parsed to a common date type. The loan status is mapped to a binary target: status A (finished, no problems) and C (running, no problems) become good loans, while status B (finished with problems) and D (running with problems) become defaults. The resulting label is imbalanced at roughly one default for every nine good loans, which shapes the rest of the methodology.

3.3.3 Feature Engineering

For every loan, the transaction table is filtered to records dated strictly before the loan date. The filtered history is aggregated into 13 behavioural features that describe activity volume, cash flow, balance level, and account history. Loan amount, duration, and payment are taken from the loan record. Age, gender, and district are taken from the client and account records. The feature table is written once and reused by all later notebooks, so the modelling code cannot accidentally see transactions after the loan date.

3.3.4 Model Training and Evaluation

The modelling notebook reads the feature table, creates a stratified train/test split, and trains the six classifiers described in Chapter 2. Each classifier is run once with its default hyperparameters and once with SMOTE applied inside the pipeline. Hyperparameters are then searched on the training split only. The tuned and untuned configurations are compared with repeated stratified cross-validation, and the selected configuration is refit on the full training split.

3.3.5 Artifact Generation and Serving

The modelling stage writes metrics tables, curve data, confusion matrices, feature importances, the fitted model, and out-of-fold risk scores. The pattern mining script writes itemsets and rules. The Streamlit application reads these files and presents four pages: model comparison, an explanation of the selected model, a review queue, and the pattern mining results. The review queue is a prioritisation aid, not an automatic approval or rejection system.

3.4 Input and Output Specification

The pipeline has two interfaces. The first is the file boundary between stages: each stage reads the artifact written by the previous stage and writes a new one. The second is the model boundary: a loan is passed to the fitted pipeline as a vector of 18 numeric features, and the pipeline returns a score. Table 3.2 lists both.

Table 3.2: Inputs and outputs at each stage of the pipeline.

Stage	**Input**	**Output**
Data assembly	`loan.csv`, `trans.csv`, `client.csv`, `account.csv`, `disp.csv`	Clean loan base table with 682 rows and the account and client keys.
Feature engineering	Loan base table and transactions dated before each loan	`data/loan_features.csv`: 682 rows, 27 columns, 18 model features.
Training	Feature table, fixed seed 42, and the stratified train/test split	Fitted pipeline `best_GradientBoosting_baseline_tuned.pkl`; metric, tuning, and curve tables.
Scoring	Fitted pipeline and one loan feature vector	Default score and rank for the loan; 682 out-of-fold scores in `loan_risk_dashboard.csv`.
Presentation	Exported CSV and PNG artifacts	Streamlit pages for comparison, explanation, review queue, and patterns.

At the model boundary, the input is the 18 features listed in Table 4.4. The output is a continuous score from `predict_proba`, treated as a rank rather than a calibrated probability. A binary label is produced only after a threshold is applied, and the review queue deliberately

uses the rank so that no single cutoff is presented as an approval rule. The five raw CSV files are the only external inputs; there is no database, API, or manual data entry in the pipeline.

3.5 Leakage Controls

Leakage is the main methodological risk in a project that combines account history with a future outcome. Four controls are applied throughout.

1. **Time cutoff.** A transaction enters a loan's feature vector only if its date is strictly earlier than the loan date. The comparison is strict, so transactions booked on the same day as the loan are excluded.

2. **Pipeline-contained preprocessing.** Scaling and SMOTE are steps inside a scikit-learn pipeline. During cross-validation they are fitted on the training fold and applied to the validation fold, never fitted on the full dataset before the split.

3. **Training-only tuning.** The randomized search sees only the training partition. The held-out test partition is used once, for the final comparison table, and does not influence the choice of model or hyperparameters.

4. **Target-free segmentation.** The KMeans segmentation used in the exploratory analysis is fitted on behavioural and loan variables with the default label excluded. Segment default rates are computed after clustering and are therefore an honest description of the groups, not an input to them.

The transaction history is also left-censored in a way that should be stated plainly. The extract begins in 1993, so a loan granted early in the period has a shorter observable history than a loan granted in 1998. The feature `months_active` records the span between the first observed transaction and the loan date, which lets the model distinguish a genuinely quiet account from one with little recorded history.

3.6 Evaluation Protocol

The evaluation follows four steps.

1. **Held-out comparison.** A stratified 75/25 split produces 511 training loans and 171 test loans, with 57 and 19 defaults respectively. All twelve model/setting combinations are

trained on the training partition and scored on the test partition. This gives a first, readable comparison under default hyperparameters.

2. **Hyperparameter search.** For each model and imbalance setting, `RandomizedSearchCV` samples up to 30 parameter combinations and scores them with F1 on an inner five-fold stratified cross-validation of the training partition. F1 is used because accuracy is dominated by the majority class at an 11% default rate.

3. **Repeated cross-validation.** The default and tuned versions of all twelve configurations are evaluated with `RepeatedStratifiedKFold(5, 10)`: five folds repeated ten times, giving 50 F1 estimates per configuration. The mean and standard deviation are reported. This step is the basis for selecting the final model, because a single 171-loan test set is too small to separate close contenders.

4. **Final refit and review scores.** The selected configuration is refit on the full training partition and saved. Separately, five-fold out-of-fold probabilities are generated for all 682 loans for the review queue. Those probabilities are out-of-sample by construction and are not the resubstitution scores of the final model.

Average precision and ROC-AUC are reported alongside F1. Accuracy and precision are included for completeness, but they are not used for selection: a model can reach high accuracy by predicting the majority class, and precision alone says nothing about how many defaults are missed.

3.7 Reproducibility

The analysis is organised as five notebooks that are executed in order:

1. `01_preprocessing.ipynb`: loads and cleans the source tables, resolves the owner relationship, and constructs the target.

2. `02_features.ipynb`: builds the pre-loan behavioural features and writes `data/loan_features.csv`.

3. `03_eda.ipynb`: produces the exploratory figures and summary statistics.

4. `04_modeling.ipynb`: runs the single-split comparison, the hyperparameter search, the repeated cross-validation, the final refit, and the out-of-fold scores.

5. `05_segmentation.ipynb`: clusters loans without the target and profiles the resulting segments.

A fixed random seed of 42 is used for the train/test split, the KMeans initialisation, and the randomized searches. The notebooks contain assertion cells for row counts, class counts, missing values, and the absence of post-loan transactions in the feature table. These checks turn silent data drift into a visible failure. The pattern mining step is kept in `tools/pattern_mining.py` because it is a short, self-contained script rather than an interactive analysis.

The next chapter describes the data preparation in detail, including the exact feature definitions and the checks applied before modelling.

Chapter 4

Data Preprocessing and Feature Engineering

The quality of a credit-risk model depends on what the feature table allows the model to see. This chapter documents the path from the five source tables to the 682-row modelling table, with particular attention to the time boundary around each loan and to the checks that were run before any classifier was trained.

4.1 Source Tables and Join Logic

The project uses all five PKDD'99 tables. The loan table is the anchor: it contains one row per loan, the account identifier, the grant date, the amount, the duration, the monthly payment, and the status that defines the target. The account table supplies the district and the account opening date. The client table supplies the birth number, which is used to derive age and gender. The disposition table connects clients to accounts; only owner dispositions are kept, because a disbursent is authorised to operate an account but is not the borrower. The transaction table supplies the behavioural history.

The loan-to-account and loan-to-client links are many-to-one after the owner filter, while the transaction history is one-to-many. The transaction table is not joined row by row to the loan table. Instead, each loan is processed against the subset of transactions belonging to its account, which avoids a quadratic expansion of the data and makes the pre-loan cutoff explicit.

After the joins and the owner filter, the loan table retains 682 rows. This is the population used in every later step. The transaction table contributes 1,056,320 records in total, of which 405,083 are credits (PRIJEM), 634,571 are debits (VYDAJ), and 16,666 are cash withdrawals (VYBER). The withdrawal features group the debit and cash-withdrawal types together.

4.2 Cleaning and Validation

Table 4.1 lists the checks performed on the assembled data. The checks are deliberately simple and are repeated as assertions in the notebooks, so a change in an input file fails loudly rather than silently changing a reported number.

Table 4.1: Data checks applied before feature construction.

Check	Result	Action
Duplicate loan rows	None found	No rows removed.
Date parsing	All %y%m%d values parsed	Invalid formats would raise an error.
Client age range	0 to 100 years	No age outliers requiring removal.
Missing values in modelling columns	None	No imputation needed.
Prior transactions per loan	At least 2	Every loan has usable history.
Owner disposition link	4,500 owner rows	Disbursent rows excluded.
Unused transaction columns	183k to 783k nulls	Left unchanged; not used as features.
Class balance	76 defaults in 682 loans	Handled by metrics and SMOTE experiments.

The null counts in the unused transaction columns are large because fields such as `operation`, `k_symbol`, `bank`, and `account` are only populated for particular transaction types. These

columns are not part of the feature set, so imputing them would add noise without adding information. They are retained in the raw extract for traceability.

4.3 Class Labels and Target Definition

The loan status is mapped to a binary label. Status A means the loan finished without problems and status C means it is running without problems; both are treated as good. Status B means the loan finished with problems and status D means it is running with problems; both are treated as default. The mapping collapses the repayment outcome into the distinction that matters for credit risk: whether the borrower ran into difficulty.

Table 4.2: Target definition and class balance.

Class	Loans	Share
Good (A or C)	606	88.86%
Default (B or D)	76	11.14%
Total	682	100.00%

The 11.14% base rate is the reference point for every precision, recall, and lift statement in the report. A classifier that predicts "good" for every loan would be right 88.86% of the time and would catch none of the 76 defaults. This is why accuracy is not used to select a model.

4.4 Pre-Loan Behavioural Features

For each loan, the transaction history is cut at the grant date. Only records with `trans.date < loan.date` are used. The cutoff is strict: a transaction booked on the same day as the loan does not enter the feature vector, because the booking time relative to the grant is not known with enough precision.

The filtered history is summarised into 13 features. Table 4.3 gives their definitions. The features fall into four groups: activity counts, cash-flow totals, transaction-size statistics, and balance history.

Table 4.3: The 13 pre-loan behavioural features. All values are computed from transactions dated strictly before the loan grant date.

Feature	Definition
Activity	
`n_txn`	Number of prior transactions.
`n_deposit`	Number of credit transactions (PRIJEM).
`n_withdrawal`	Number of debit or cash-withdrawal transactions (VYDAJ, VYBER).
Cash flow	
`sum_deposit`	Total amount credited before the loan.
`sum_withdrawal`	Total amount debited or withdrawn before the loan.
Transaction size	
`avg_amount`	Mean transaction amount.
`std_amount`	Population standard deviation of transaction amounts.
Balance and history	
`avg_balance`	Mean running balance after each prior transaction.
`min_balance`	Lowest running balance observed before the loan.
`max_balance`	Highest running balance observed before the loan.
`balance_at_loan`	Running balance after the most recent prior transaction.
`months_active`	Months between the first prior transaction and the loan date, using 30.44 days per month and a floor of one day.
`txn_per_month`	Prior transaction count divided by `months_active`.

The balance features are cumulative in the sense that each transaction's recorded balance already reflects earlier activity. They are not independent observations, but they answer a question that the raw amounts cannot: whether the account was under strain shortly before the loan was granted. The minimum balance and the balance at the loan date turn out to be the two most useful behavioural signals in the fitted models.

The history length deserves a caveat. A loan granted in 1993 may have only a few months of observed transactions, while a loan granted in 1998 may have several years. `months_active` and `txn_per_month` reduce, but do not remove, this left-censoring. The feature table records the span so the model can separate a low-activity account from a short observation window.

4.5 Loan Terms and Demographics

Three loan terms are added from the loan record: the requested `amount`, the `duration` in months, and the monthly `payments`. These are known at the grant date and are legitimate predictors. They also carry the repayment burden: a large loan relative to the account balance is a different proposition from a small loan with the same history.

Two demographic features are derived from the client birth number. The birth number encodes the date of birth in its first six digits, with the month field offset by 50 for women. Gender is therefore derived as female when the two-digit month is greater than 50, and male otherwise. This yields 2,645 female clients and 2,724 male clients in the full client table. Age is computed against the reference year 1999, which is the end of the observation window, as $1999 - (year of birth + 1900)$.

`gender_bin` is coded as 1 for female and 0 for male. The district identifier is kept in the feature table for descriptive profiling but is not one of the 18 modelling features. The model matrix is deliberately compact: it contains the loan terms, the 13 behavioural features, age, and gender, and leaves out the high-cardinality district identifier.

4.6 Final Feature Table

The output of the feature stage is `data/loan_features.csv`, with 682 rows and 27 columns. The extra columns beyond the 18 modelling features are identifiers, dates, the raw status, and intermediate fields used for checks and joins. Table 4.4 groups the 18 features that enter the classifiers.

Table 4.4: The 18 features used by the classifiers, by group.

Group	Features
Loan terms (3)	`amount`, `duration`, `payments`
Activity (3)	`n_txn`, `n_deposit`, `n_withdrawal`
Cash flow (2)	`sum_deposit`, `sum_withdrawal`
Transaction size (2)	`avg_amount`, `std_amount`
Balance (4)	`avg_balance`, `min_balance`, `max_balance`, `balance_at_loan`
History (2)	`months_active`, `txn_per_month`
Demographics (2)	`age`, `gender_bin`

The table has no missing values and no duplicate loan identifiers. The average loan has about 80 prior transactions over roughly 13 months, an average balance near 43,600, and a balance at the loan date near 42,550. Those averages hide a wide spread: the balance distribution runs from overdrafts near $-17{,}000$ to balances above 100,000.

4.7 Transformation and Train/Test Split

Tree-based models do not require scaling, but the SVM and KNN do. Scaling is therefore placed inside each model's pipeline rather than applied once to the whole table. This keeps the scaler's mean and variance inside the training fold during cross-validation. SMOTE, when used, is also a pipeline step and is applied after scaling. The resampling is never performed before the split.

The data is divided with a stratified 75/25 split and a fixed seed of 42. Stratification preserves the default rate in both partitions. Table 4.5 shows the result.

Table 4.5: Stratified train/test split.

Partition	Loans	Good	Default	Default rate
Training	511	454	57	11.15%
Test	171	152	19	11.11%

The test partition contains only 19 defaults. A single split therefore has limited power to separate models whose F1 differs by a few points. This is the reason the project adds repeated cross-validation on the training partition and treats the test table as a descriptive check rather than as the final arbiter.

4.8 Data Quality Observations

Four properties of the feature table shaped the modelling choices.

- **Skew.** The money features are right-skewed. A small number of accounts have very large turnover, while most sit in a narrow band. Tree models are insensitive to monotone transformations, and the scaled models use standardisation, so no log transform was applied. The skew remains visible in the exploratory plots and is one reason the median is reported alongside the mean for balance comparisons.

- **Overdrafts.** Minimum balances reach about $-17{,}000$. These are genuine overdrafts, not data errors, and they are informative: a negative minimum balance is the strongest single behavioural signal in the pattern mining step.

- **Collinearity.** `sum_deposit` and `sum_withdrawal` correlate at 0.997. Both are kept for interpretation, but their near-duplication means their individual importance scores should not be read as independent effects. The loan amount and monthly payment also move together, as expected from the repayment formula.

- **Class imbalance.** With 76 defaults, the minority class is small in absolute terms. Resampling can change the balance but cannot create new information. The SMOTE experiments in Chapter 6 are therefore treated as an empirical question, not as an assumed improvement.

The next chapter uses the cleaned feature table for exploratory analysis. The goal there is not to select a model but to understand which behaviours separate good and defaulted loans before any classifier is fitted.

Chapter 5

Exploratory Data Analysis and Visualizations

The exploratory analysis asks a narrow question: which observable behaviours at the grant date separate loans that were repaid from loans that ran into difficulty? The analysis is descriptive. No feature is selected or transformed on the basis of the target in a way that later leaks into the model. The figures in this chapter are produced from the same 682-row feature table used for modelling.

5.1 Class Distribution

Figure 5.1 shows the target distribution. The dataset contains 606 good loans and 76 defaults, an 11.14% default rate. The imbalance is moderate in absolute terms but severe enough that a majority-class predictor would reach 88.86% accuracy while identifying no defaults. Every later evaluation therefore reports recall, F1, average precision, and ROC-AUC alongside accuracy.

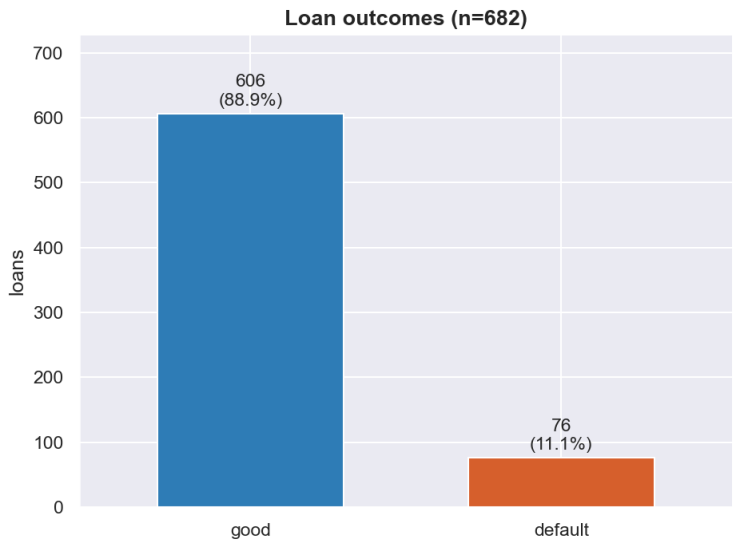


Figure 5.1: Distribution of the binary target: 606 good loans and 76 defaults (11.14%).

5.2 Features Associated with Default

Figure 5.2 ranks the numeric features by their Pearson correlation with the default label. The strongest associations are negative and involve account balance: `min_balance` at -0.26, `balance_at_loan` at -0.20, and `avg_balance` at -0.16. Borrowers who defaulted tended to have lower balances before the loan, and the lowest point of their balance history was further below zero than for good borrowers.

On the loan side, `amount` correlates at $+0.17$ and `payments` at $+0.18$: larger loans and the higher monthly payments that come with them are associated with default. Age and gender are close to zero ($+0.01$ and $+0.02$), which is an early indication that the demographic variables carry little signal in this dataset.

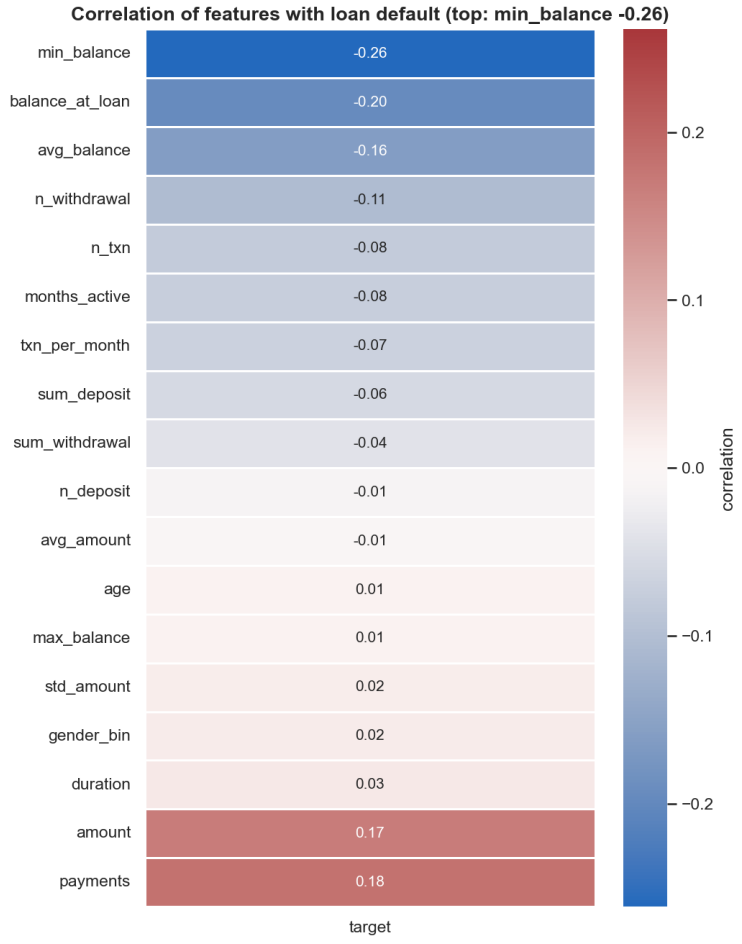


Figure 5.2: Correlation of each numeric feature with the default label. Balance features are negative; loan size and payment are positive.

Correlation is a linear summary and can understate a relationship that is concentrated in a

tail. The next two sections look at distributions rather than coefficients.

5.3 Balance Separation

Figure 5.3 compares the distributions of the main behavioural features for good and defaulted loans. The medians tell a consistent story. The balance at the loan date has a median of about 39,500 for good loans and 22,000 for defaults. Average balance is about 44,100 for good loans and 37,000 for defaults. The gap is not a handful of extreme accounts: the default distribution sits lower across its middle.

Transaction frequency barely separates the classes. Both groups transact at roughly 5.6 to 5.7 times per month. A busy account is not automatically a safe account. What matters more is the level of the balance and whether it was falling before the loan.

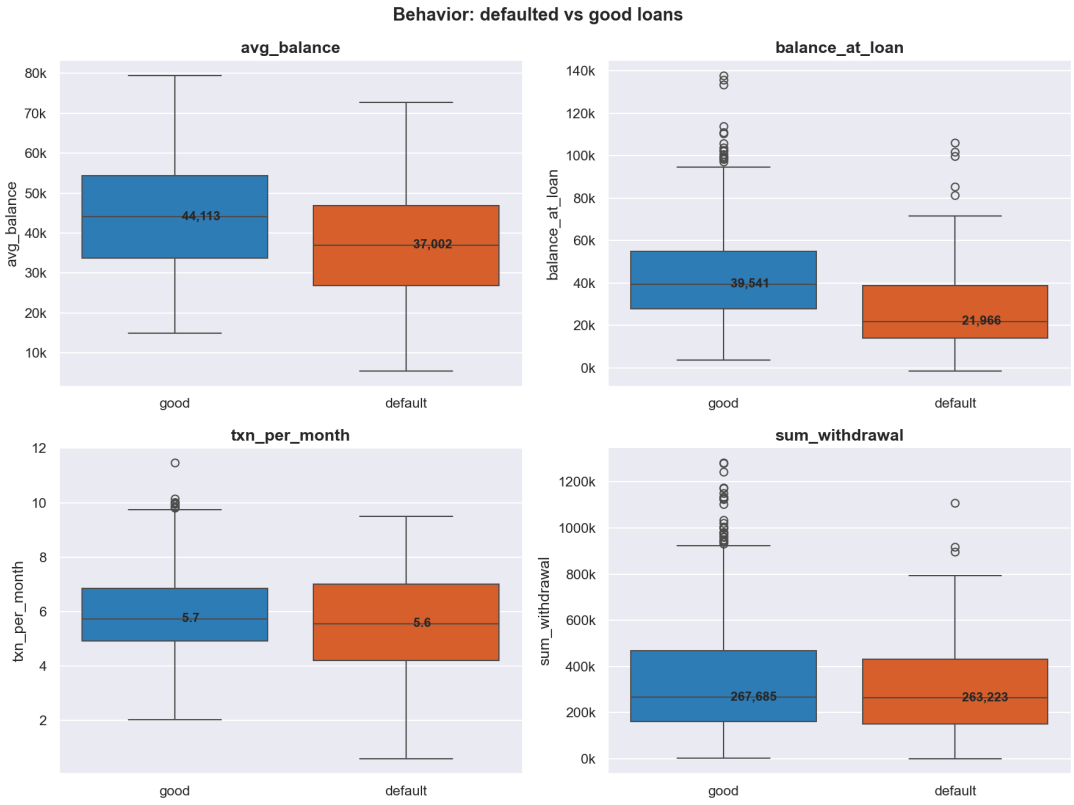


Figure 5.3: Box plots of selected behavioural features by class. The balance distributions separate more clearly than transaction frequency.

5.4 Age and Gender

Figure 5.4 shows the age distribution split by outcome. Defaults appear across the full age range, with no concentration in a particular decade. The visual impression matches the near-zero correlation: age is not a useful discriminator on its own.

The gender split is similarly weak. Female borrowers default at 11.8% and male borrowers at 10.5%, a difference of 1.3 percentage points on a small minority class. With 76 defaults in total, that difference is well within sampling noise. Gender is kept as a feature for completeness, but the report does not build a business rule around it.

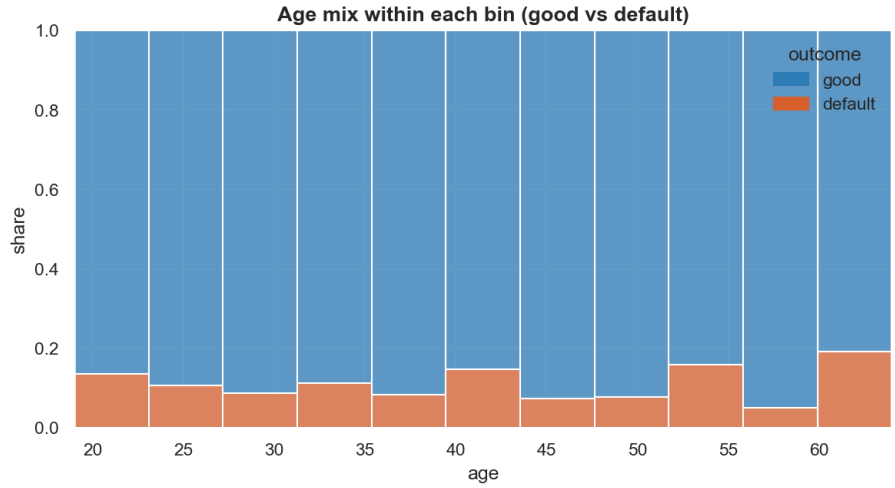


Figure 5.4: Age distribution by outcome. Defaults occur across all ages rather than in a single band.

5.5 Collinearity and Redundancy

Figure 5.5 shows the full correlation matrix of the numeric features. Two clusters stand out. The first is the cash-flow pair: `sum_deposit` and `sum_withdrawal` correlate at 0.997, which is close to a duplicate. The second is the loan-burden pair: `amount` and `payments` move together, as they must under a fixed rate and duration.

The balance features form a softer cluster. `avg_balance`, `min_balance`, `max_balance`, and `balance_at_loan` are related but not interchangeable; the minimum captures a different part of the distribution than the last observed value. This is why all four are retained, even though their importance scores will partly overlap.

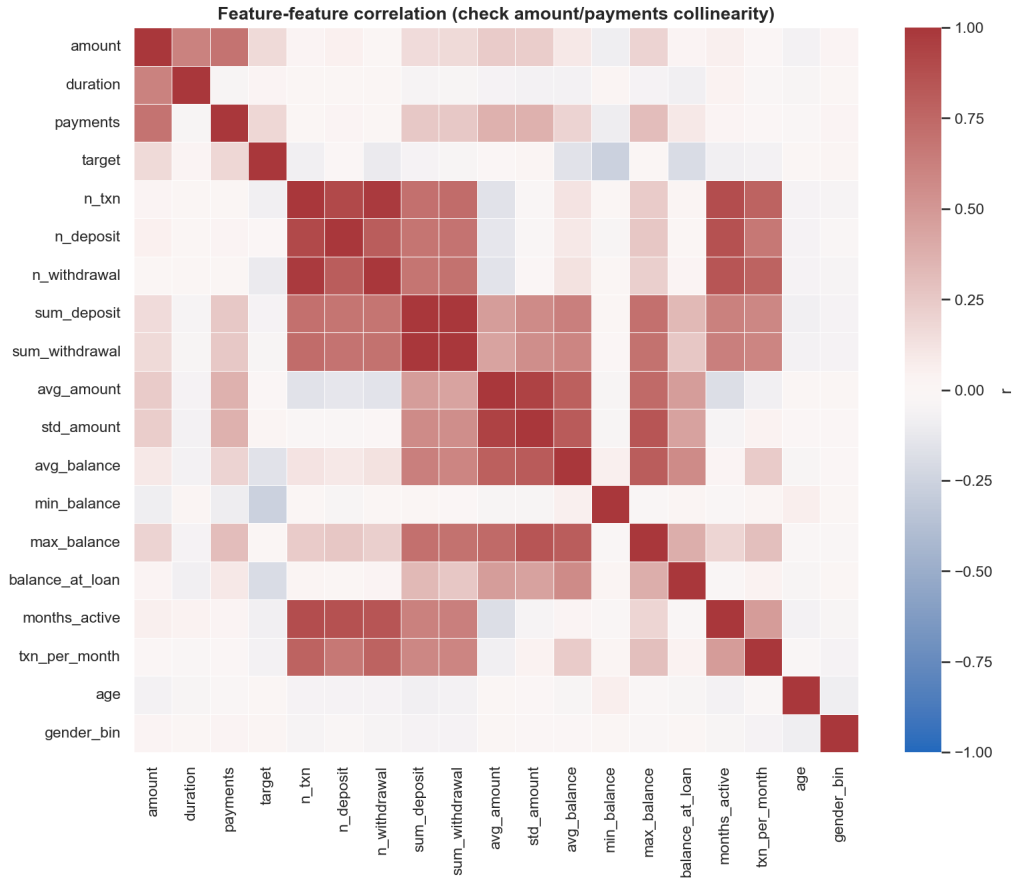


Figure 5.5: Correlation matrix of the numeric features. The cash-flow pair is nearly duplicated, and loan amount tracks monthly payment.

The redundancy has a practical consequence for interpretation. When two features carry almost the same information, a tree can split on either one, and the reported importance can move between them across runs. The feature importance discussion in Chapter 6 therefore groups related variables rather than treating each score as a separate causal effect.

5.6 Loan Size Relative to Balance

The two strongest themes in the univariate analysis are balance level and loan size. Figure 5.6 combines them. Loans are split at the median pre-loan balance of about 38,500 and the median loan amount of about 116,900. The four cells have very different default rates.

Table 5.1: Default rate by pre-loan balance and loan amount, split at the respective medians.

Balance	Loan amount	Default rate
High	Low	4.9%
High	High	6.7%
Low	Low	10.1%
Low	High	23.3%

The highest-risk cell is the borrower with a below-median balance asking for an above-median loan. Its default rate of 23.3% is more than double the base rate and roughly five times the rate in the high-balance, low-amount cell. The pattern is intuitive: the loan is large relative to the buffer the account has shown. It also suggests a simple screening rule that does not require a model, which is useful as a sanity check on whatever the classifiers learn.

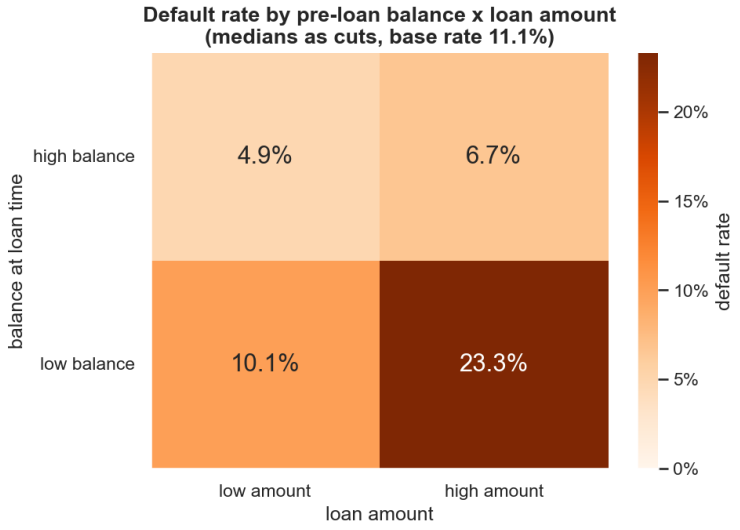


Figure 5.6: Default rate by balance and loan amount. The low-balance, high-amount quadrant has the highest risk.

5.7 Behavioural Segments

A KMeans clustering with two groups was fitted on the behavioural and loan variables, with the target excluded. The silhouette score of 0.29 is modest, so the segments should be read as a coarse summary rather than as natural kinds. Figure 5.7 shows the separation in the balance and activity space, and Table 5.2 gives the profiles.

Table 5.2: Profiles of the two behavioural segments. The target was not used to form the clusters.

Segment	Loans	Median txns	Mean balance	Txns/month	Default rate
0	421	54	37,200	5.2	12.4%
1	261	116	52,400	6.9	9.2%

Segment 0 is the larger, quieter group: fewer transactions, lower average balance, and a default rate of 12.4%. Segment 1 is more active and holds a higher balance, with a default rate of 9.2%. The direction is sensible, but the overlap is substantial. Activity alone does not separate the classes cleanly, which is consistent with the weak univariate correlation of `txn_per_month`. The segment label is useful for describing the portfolio, not for deciding a loan.

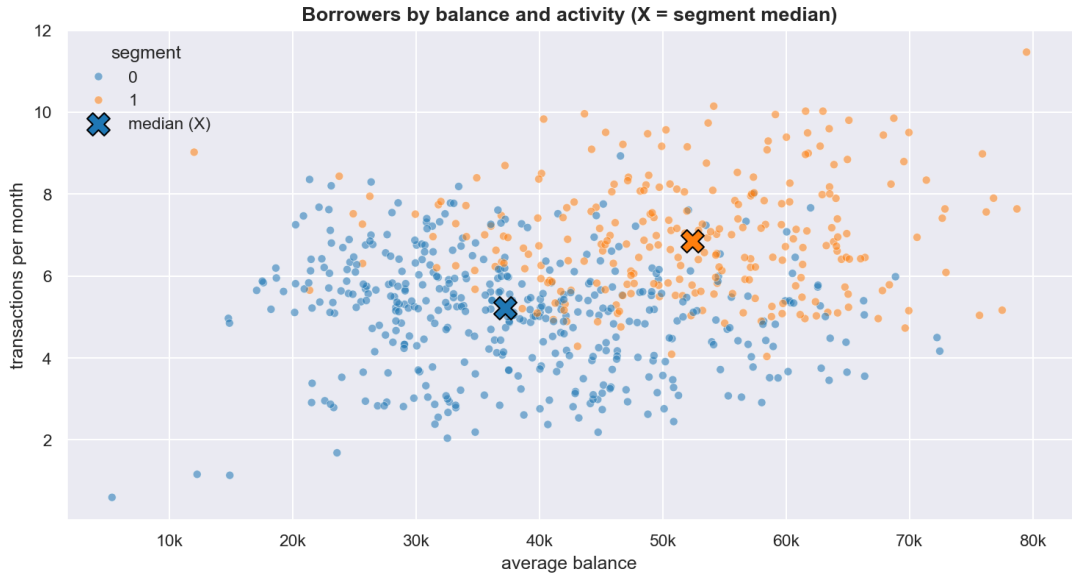


Figure 5.7: Loans in the two behavioural segments, plotted in the activity and balance space.

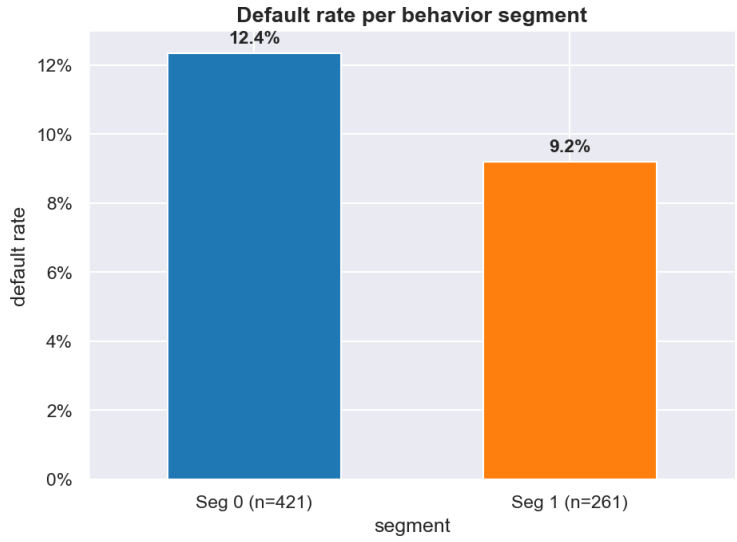


Figure 5.8: Default rate by segment. The quieter, lower-balance segment has the higher rate, but the difference is modest.

5.8 Summary of Findings

The exploratory work supports five conclusions that carry into the modelling chapter.

- Balance history is the main behavioural axis. `min_balance` and `balance_at_loan` have the strongest correlations with default, and their distributions separate visibly.
- Loan size matters in combination with balance. The low-balance, high-amount quadrant reaches a 23.3% default rate, far above the 11.14% base rate.
- Age and gender carry little signal. Their correlations are near zero and the age distribution shows no high-risk band.
- Activity volume is weak on its own. Transaction frequency is similar across classes, and the two clusters differ by only about three percentage points in default rate.
- Collinearity is present but manageable. The cash-flow pair is nearly duplicated, and the balance features overlap; interpretation should focus on groups of related variables.

These findings set expectations for the classifiers. A model should improve on the base rate mainly by using balance and loan burden, should not gain much from demographics, and should be judged on its ability to catch the small default class rather than on overall accuracy.

Chapter 6

Classification Models and Performance Evaluation

This chapter compares six classifiers under two imbalance strategies. It first describes the configurations and the training protocol, then reports the performance metrics, the held-out test results, the repeated cross-validation results, the ranking curves, the feature importance analysis, and the pattern matching results. The chapter closes with an interpretation of what the numbers do and do not support.

6.1 Classifiers and Configurations

Six classifiers were chosen to cover different inductive biases: a single tree, a probabilistic baseline, a bagged ensemble, a margin-based method, an instance-based method, and a boosted ensemble. Table 6.1 lists them and the reason each was included.

Table 6.1: Classifiers evaluated in the project.

Classifier	**Implementation and role**
Decision Tree	`DecisionTreeClassifier`. A single axis-aligned tree. It is the most interpretable model in the set and provides a non-ensemble reference point.
Naive Bayes	`GaussianNB`. A fast probabilistic baseline that assumes conditional independence of the features.
Random Forest	`RandomForestClassifier`. A bagged ensemble of deep trees. It captures interactions and is robust to irrelevant features.
Support Vector Machine	`SVC`, wrapped in `CalibratedClassifierCV`. A margin-based classifier with RBF and linear kernels. It requires scaled inputs.
K-Nearest Neighbours	`KNeighborsClassifier`. An instance-based classifier that labels a loan by the majority of its nearest neighbours. It also requires scaled inputs.
Gradient Boosting	`GradientBoostingClassifier`. A sequential ensemble that fits each tree to the residual of the previous ones.

Each classifier is evaluated under two imbalance settings. The baseline arm trains on the natural class distribution. The SMOTE arm applies Synthetic Minority Over-sampling Technique inside the pipeline, so the minority class is oversampled to parity within each training fold. The combination gives six models times two settings, or twelve configurations.

The SVM is wrapped in a calibrator because the probability estimates of `SVC` were deprecated in recent scikit-learn releases. The calibrator is fitted inside the pipeline and does not see the validation fold.

6.2 Training Pipeline

The pipeline for each configuration has four steps: optional scaling, optional SMOTE, the classifier, and prediction. Scaling is included for SVM and KNN and omitted for the tree models, which do not need it. The order matters: SMOTE is applied after scaling, so the synthetic points are generated in the same coordinate system as the real ones, and both steps are fitted only on the training portion of each split.

The protocol has three stages.

1. **Single split.** All twelve configurations are trained on the 511-loan training partition with default hyperparameters and scored on the 171-loan test partition.

2. **Hyperparameter search.** For each configuration, `RandomizedSearchCV` samples up to 30 parameter combinations and scores them with F1 on an inner five-fold stratified cross-validation of the training partition. The search never touches the test partition. Naive Bayes has 9 candidates and KNN has 24 because their grids are small.

3. **Repeated cross-validation.** The default and tuned versions of all twelve configurations are evaluated with five-fold stratified cross-validation repeated ten times. This produces 50 F1 estimates per configuration and is the table used to select the final model.

The selection metric is mean F1 from the repeated cross-validation. Average precision and ROC-AUC are reported alongside it, but they are not used to re-rank the models after the fact. Fixing the criterion before seeing the results avoids choosing whichever metric happens to favour a preferred model.

6.3 Hyperparameter Tuning

Table 6.2 lists the best configuration found for each model and imbalance setting, the number of candidates evaluated, the inner cross-validation F1, and the selected parameters. The search is deliberately modest: 30 candidates per configuration is enough to show whether tuning helps, without turning a 511-row training set into a long optimisation exercise.

Table 6.2: Hyperparameter search results, ranked by inner cross-validation F1.

Model	Arm	Cand.	Inner F1	Best parameters
Gradient Boosting	Baseline	30	0.582	`subsample=0.7; n_estimators=300; max_depth=1; learning_rate=0.05`
Random Forest	SMOTE	30	0.576	`n_estimators=200; min_samples_leaf=5; max_features=sqrt; max_depth=20`
Gradient Boosting	SMOTE	30	0.538	`subsample=0.7; n_estimators=300; max_depth=2; learning_rate=0.05`
SVM	SMOTE	30	0.530	`kernel=linear; gamma=0.01; C=100`
Random Forest	Baseline	30	0.519	`n_estimators=200; min_samples_leaf=1; max_features=log2; max_depth=8`
Decision Tree	SMOTE	30	0.510	`min_samples_leaf=2; max_depth=12; criterion=gini; class_weight=None`
SVM	Baseline	30	0.503	`kernel=rbf; gamma=0.01; C=0.1`
Decision Tree	Baseline	30	0.492	`min_samples_leaf=5; max_depth=3; criterion=gini; class_weight=None`
KNN	SMOTE	24	0.385	`weights=distance; p=2; n_neighbors=21`
Naive Bayes	Baseline	9	0.371	`var_smoothing=1e-11`
Naive Bayes	SMOTE	9	0.321	`var_smoothing=1e-11`
KNN	Baseline	24	0.229	`weights=uniform; p=2; n_neighbors=3`

The strongest inner-CV result belongs to Gradient Boosting without resampling: 0.582 with a shallow model of 300 stumps, a learning rate of 0.05, and 70% subsampling. The selected depth of one means each boosting round adds a single split, which keeps the model close to an additive model and limits overfitting on 511 rows.

The tuning curves in Figure 6.1 show F1 against the key hyperparameter for the two strongest families. The curves are flat over much of the range and then fall sharply. That shape is a warn-

ing against reading too much into a small difference between neighbouring settings: the surface is noisy, and the inner CV has only 57 defaults to work with.

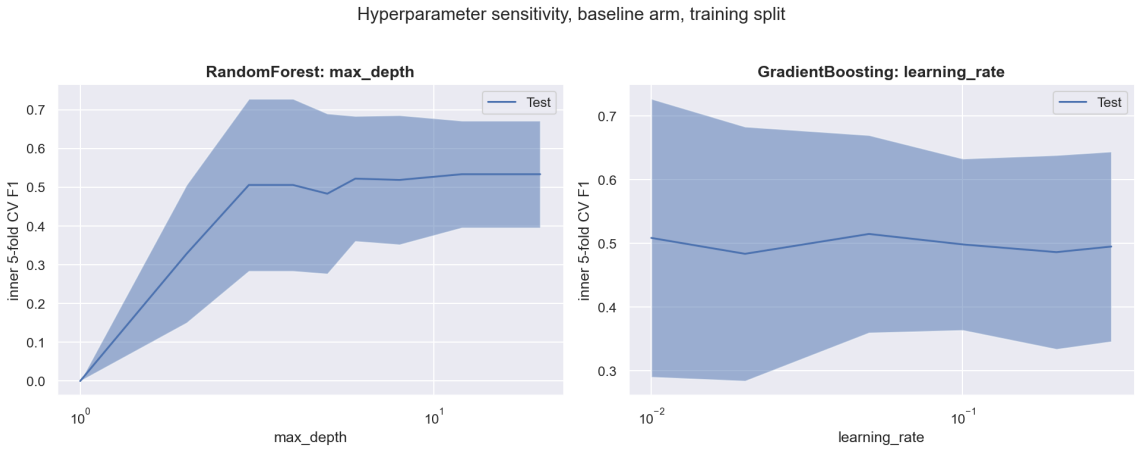


Figure 6.1: Hyperparameter sensitivity for the baseline arm on the training split. F1 is flat over a broad region and falls at the extremes.

6.4 Performance Metrics and Held-Out Test Results

Table 6.3 reports the single-split results for all twelve configurations with default hyperparameters. The test partition has 171 loans, 19 of which defaulted. A single default therefore moves recall by about 5.3 percentage points, which is the scale against which differences should be read.

Table 6.3: Held-out test performance with default hyperparameters (171 loans, 19 defaults).

Model	Arm	Acc.	Prec.	Rec.	F1	AP	AUC
Random Forest	Baseline	0.924	0.875	0.368	0.519	0.582	0.856
Decision Tree	Baseline	0.895	0.529	0.474	0.500	0.309	0.711
Gradient Boosting	Baseline	0.912	0.700	0.368	0.483	0.583	0.816
SVM	Baseline	0.918	0.857	0.316	0.462	0.551	0.771
Naive Bayes	Baseline	0.807	0.150	0.158	0.154	0.171	0.634
KNN	Baseline	0.895	1.000	0.053	0.100	0.220	0.668
Gradient Boosting	SMOTE	0.889	0.500	0.421	0.457	0.495	0.741
Decision Tree	SMOTE	0.883	0.471	0.421	0.444	0.262	0.681
Random Forest	SMOTE	0.895	0.538	0.368	0.438	0.532	0.826
SVM	SMOTE	0.854	0.375	0.474	0.419	0.385	0.726
KNN	SMOTE	0.743	0.195	0.421	0.267	0.156	0.645
Naive Bayes	SMOTE	0.591	0.108	0.368	0.167	0.155	0.563

The Random Forest has the highest accuracy (0.924) and precision (0.875), but it catches only 7 of the 19 defaults (recall 0.368). The Decision Tree catches 9 of 19 (recall 0.474) at a lower precision, which gives it the second-best F1. The Gradient Boosting sits between them with an F1 of 0.483 and the highest average precision of the baseline arm (0.583).

The KNN baseline illustrates why precision alone is misleading. It reports a precision of 1.000 because it predicts a single default and gets it right, but its recall is 0.053. A credit team could not use a model that reviews one case and misses eighteen.

SMOTE raises recall for most models but lowers precision by more. The Gradient Boosting arm moves from a recall of 0.368 to 0.421 while precision falls from 0.700 to 0.500. The net F1 is lower. The Naive Bayes SMOTE model is the clearest failure: accuracy falls to 0.591 and precision to 0.108, so the resampling has produced many false alarms without a useful gain in coverage.

Figure 6.2 shows the confusion matrices for the twelve configurations. The top row, which contains the baseline models, is dominated by false negatives for the high-precision models. The SMOTE row shifts errors toward false positives. Neither error type is free: a false negative is a missed risky loan, while a false positive is a good customer sent to manual review.

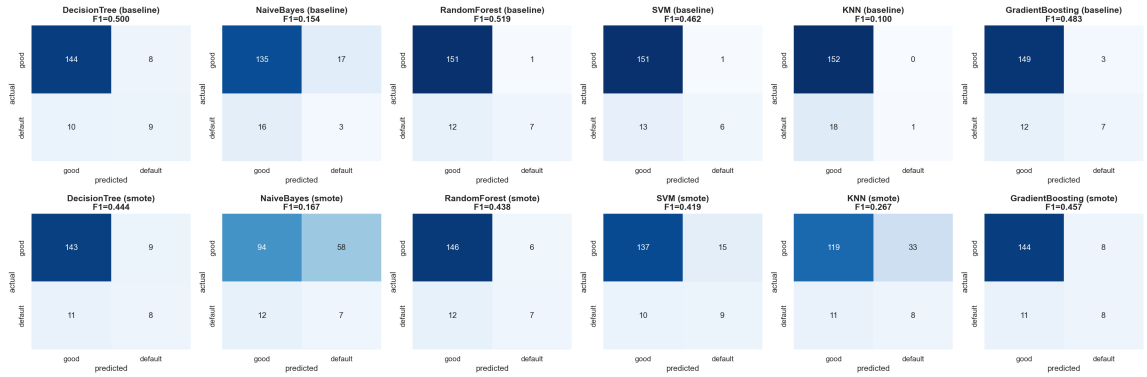


Figure 6.2: Confusion matrices on the held-out test set with default hyperparameters. The two rows show the baseline and SMOTE arms.

6.5 Validation Test: Repeated Cross-Validation

A single test set of 171 loans cannot reliably separate models whose F1 differs by two or three points. Table 6.4 therefore reports the mean and standard deviation of F1 over 50 estimates from five-fold cross-validation repeated ten times. The table is ranked by tuned F1, and it includes the default F1 for the same configuration so the effect of tuning is visible.

Table 6.4: Repeated stratified five-fold cross-validation, ten repeats (50 estimates), ranked by tuned F1. The last two columns describe the tuned configuration.

Model	Arm	F1 default	F1 tuned	AP tuned	AUC tuned
Gradient Boosting	Baseline	0.557	0.571 ± 0.120	0.654	0.864
Decision Tree	Baseline	0.491	0.557 ± 0.126	0.507	0.748
Random Forest	Baseline	0.557	0.556 ± 0.112	0.661	0.877
Random Forest	SMOTE	0.537	0.542 ± 0.103	0.638	0.872
Gradient Boosting	SMOTE	0.542	0.536 ± 0.101	0.641	0.851
SVM	Baseline	0.421	0.498 ± 0.128	0.603	0.842
Decision Tree	SMOTE	0.452	0.464 ± 0.104	0.307	0.741
SVM	SMOTE	0.436	0.463 ± 0.084	0.608	0.848
KNN	SMOTE	0.336	0.367 ± 0.065	0.367	0.768
Naive Bayes	Baseline	0.325	0.325 ± 0.132	0.356	0.769
Naive Bayes	SMOTE	0.318	0.318 ± 0.055	0.338	0.729
KNN	Baseline	0.198	0.223 ± 0.129	0.249	0.628

Gradient Boosting with the baseline arm has the highest mean tuned F1 at 0.571. Random Forest is second at 0.556 and has the best average precision (0.661) and ROC-AUC (0.877). The F1 gap between them is 0.015, while the fold-to-fold standard deviation is about 0.11. The two models are effectively tied on F1, and the Random Forest ranks positives better. The selection rule nevertheless points to Gradient Boosting because F1 was fixed as the criterion before the comparison. The report keeps Random Forest as a credible alternative rather than pretending the choice is unambiguous.

The Decision Tree is the surprise of the table. Tuning raises its F1 from 0.491 to 0.557, a gain of 0.066, and places it third. Its average precision is only 0.507, however. The tree has learned a decision boundary that captures more defaults at the cost of a less reliable ranking. This is the clearest example of why F1 and AP should be read together: they answer different questions.

The tuning gains are uneven. The SVM improves by 0.077 and the Decision Tree by 0.066, while the Random Forest does not improve at all (-0.001) and Naive Bayes is unchanged. The models with strong default settings had less room to gain, and the search may simply have confirmed that their defaults were already appropriate for this sample size.

SMOTE helps only KNN at the tuned comparison (0.223 to 0.367). For the other five models, the tuned baseline arm is at least as good as the tuned SMOTE arm. The result is consistent with the test table and with the small absolute number of defaults: synthetic minority points can smooth a boundary that is already hard to estimate, and the extra recall often arrives with a larger loss in precision. The project therefore does not recommend SMOTE as a default for this dataset.

Figure 6.3 shows the tuning gain for each configuration, with the selected model marked. The visual makes the two large gains (SVM and Decision Tree) and the flat response of the already-strong models easy to see.

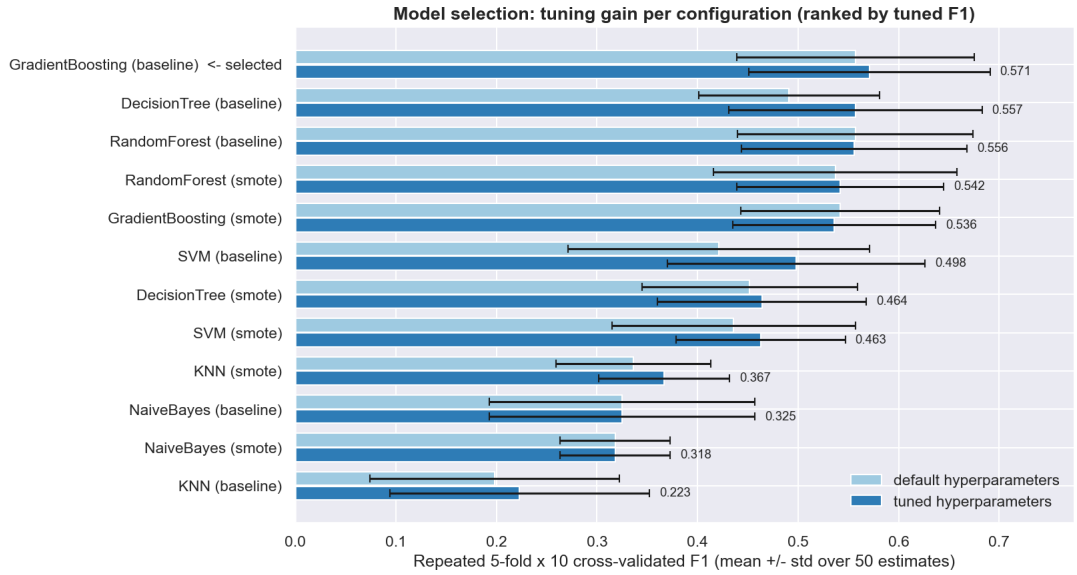


Figure 6.3: Tuning gain per configuration under repeated cross-validation. The selected Gradient Boosting baseline is marked.

6.6 Ranking Curves

Figures 6.4 and 6.5 show the ROC and precision-recall curves for the default configurations. The ROC curves of Random Forest, Gradient Boosting, and SVM sit well above the diagonal, with AUC values in the 0.77 to 0.86 range on the test set. The precision-recall curves are more sobering. At an 11.14% prevalence, the no-skill average precision is 0.111, and the best default model reaches about 0.58. That is a real improvement, but it is far from a clean separation.

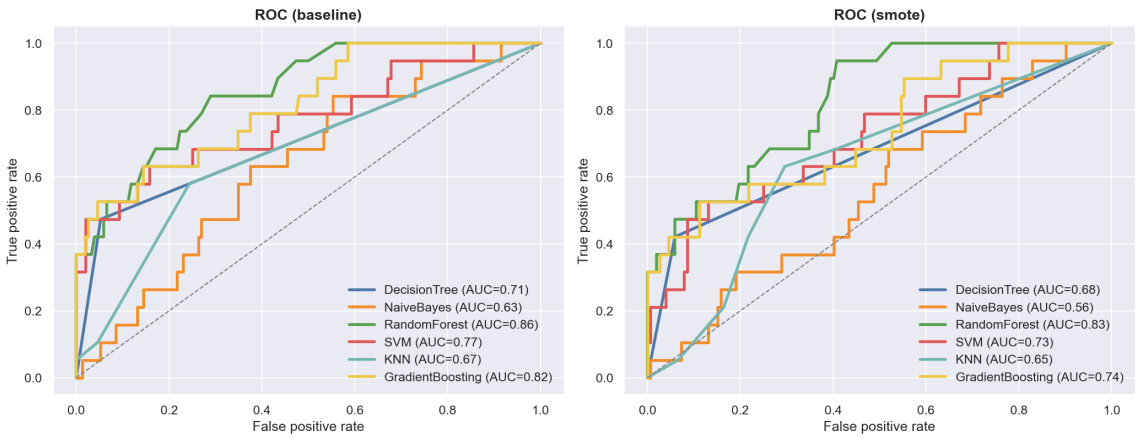


Figure 6.4: ROC curves by imbalance arm on the held-out test set, default hyperparameters.

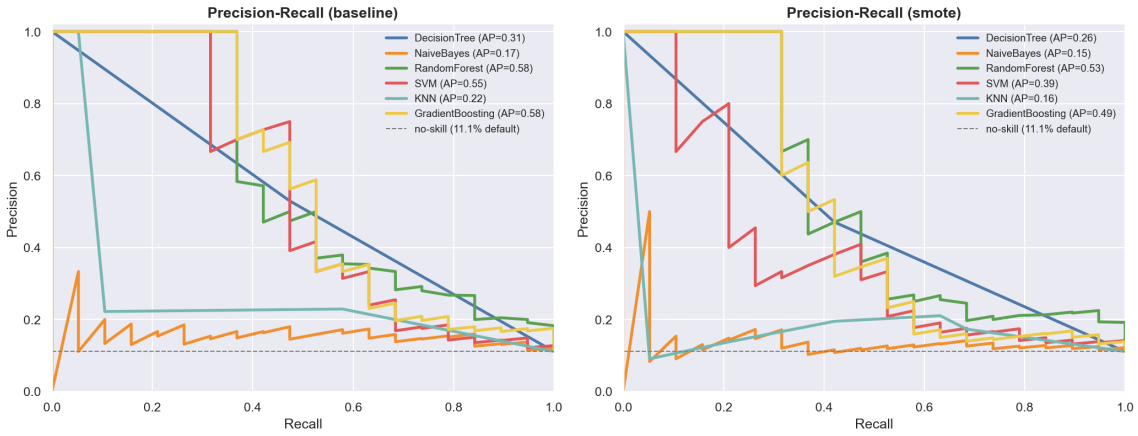


Figure 6.5: Precision-recall curves with average precision on the held-out test set, default hyperparameters. The dashed baseline is the 11.14% default rate.

The ROC view can flatter a model on imbalanced data because a large number of true negatives keeps the false-positive rate low even when many defaults are missed. The precision-recall view makes the cost visible: to reach a recall of 0.5, a model must accept a precision well below its peak. The operating threshold is therefore a business decision, not a property of the classifier.

6.7 Feature Importance

Figure 6.6 shows impurity-based importance from the tuned Random Forest. The Random Forest is used for this view because tree ensembles expose a direct importance score and the tuned forest was one of the two strongest models. The scores are not causal effects; they describe how much a feature contributed to reducing impurity across the forest.

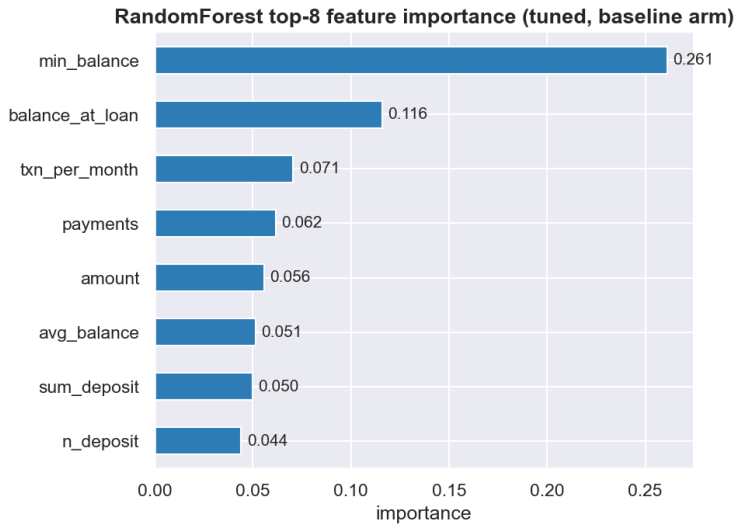


Figure 6.6: Feature importance from the tuned Random Forest. Balance features dominate; demographic features contribute little.

`min_balance` is the single most important feature at 0.261, followed by `balance_at_loan` at 0.116. The two balance features together account for more than a third of the total importance. Transaction frequency (0.071), monthly payment (0.062), loan amount (0.056), and average balance (0.051) form a second tier. Age (0.025) and gender (0.005) are near the bottom.

Two cautions apply. First, impurity importance is biased toward features with many distinct values, and all the money and count features are continuous. Second, correlated features share credit: when `sum_deposit` and `sum_withdrawal` carry almost the same information, the forest may split on either one, and their individual scores can change between runs. The grouped conclusion is safer than any single rank: balance history and loan burden carry the signal, while demographics do not.

6.8 Pattern Matching with Apriori and FP-Growth

The pattern mining step looks for simple co-occurrences between pre-loan behaviours and default. It uses Apriori and FP-Growth on one basket per account, built from the first loan on the account and transactions dated before that loan. An item records whether the account ever showed a behaviour in that window. The minimum support is 0.03, the maximum itemset length is three, the minimum confidence is 0.10, and rules are kept only if the lift is above 1.0 and the consequent is default. Redundant rules are pruned when a more general rule is at least as strong.

Both algorithms return the same 41 itemsets and the same two default-oriented rules, which is a useful consistency check. Table 6.5 reports the rules.

Table 6.5: Default-oriented association rules after redundancy pruning. Support is the share of the 682 accounts; coverage is the share of the 76 defaults.

Antecedent	Antecedent support	Confidence	Lift	Default coverage
`negative_balance`	3.96% (27/682)	100.0%	8.97	35.5% (27/76)
`type_VYBER`	54.11% (369/682)	13.0%	1.17	63.2% (48/76)

The first rule is striking: all 27 accounts that ever had a negative running balance before the first loan defaulted. The lift of 8.97 and the confidence of 1.000 make it the strongest pattern in the data. It is also a small group, so the confidence interval around 100% is wide, and the rule is univariate: it does not control for loan size, history length, or the other behaviours. It should be read as a signal worth investigating, not as a lending policy.

The second rule is weak. Accounts that used cash withdrawals (VYBER) default at 13.0%, barely above the 11.14% base rate, for a lift of 1.17. The rule covers 48 defaults because cash withdrawals are common (54.11% of accounts), not because they are dangerous. It is reported for completeness and is not actionable on its own.

The difference between the two rules is a useful summary of the whole analysis. A rare, extreme behaviour (a negative balance) is highly predictive but covers a minority of defaults. A common behaviour (cash withdrawal) covers many defaults but barely improves on the base rate. A usable system needs both: a high-precision rule for a small group, and a model that ranks the rest of the portfolio.

6.9 Discussion

The results support four conclusions.

- **The top models are close.** Gradient Boosting and Random Forest are separated by 0.015 F1 under repeated cross-validation, with fold-to-fold standard deviations around 0.11. The choice of Gradient Boosting follows the pre-specified F1 criterion, but Random Forest has better average precision and ROC-AUC and is a defensible alternative.

- **SMOTE is not a default fix.** It improves KNN and raises recall for several models, but it lowers F1 for five of six tuned configurations. With 76 defaults, synthetic samples do not add enough new information to justify the precision loss.
- **Tuning helps some models a lot and others not at all.** The SVM gains 0.077 F1 and the Decision Tree 0.066, while the Random Forest and Naive Bayes are flat. Hyperparameter search is most useful when the default model is poorly matched to the data scale or the tree depth.
- **Ranking metrics change the story.** The Decision Tree reaches a respectable F1 after tuning but has a much lower average precision than the forest. A credit team that reviews the highest-risk cases should care about the ranking, not only about the thresholded F1.

The limitations are equally important. The test set contains 19 defaults, so a single split has wide uncertainty. The cross-validation reuses the same folds for tuning and reporting, which makes the tuned means mildly optimistic; a nested cross-validation would give a cleaner estimate of the tuning procedure. The data is a 1990s Czech retail portfolio and does not represent current borrowers. The features describe account behaviour, not income, employment, or credit bureau history, so the model cannot see the most common reasons for default. Finally, the predicted scores are ranks, not calibrated probabilities. A threshold of 0.5 has no special meaning without calibration, and the review queue uses the ranking rather than a fixed probability cutoff.

6.10 Interactive Review Console

The results are exposed through a Streamlit application, started with `streamlit run app.py`. The application does not retrain models; it reads the CSV artifacts written by the notebooks. It has four pages.

1. **Model comparison.** A table of the twelve configurations with the held-out metrics, plus the ROC and precision-recall curves. This page answers the question "how do the models compare?" at a glance.
2. **Why the best model.** The tuning result, the selected parameters, and the feature importance chart. This page explains what the selected model relies on and where its limits are.

3. **Review queue.** A ranked list of loans by out-of-fold default probability, with a threshold slider. The probabilities come from five-fold out-of-fold predictions over all 682 loans, not from resubstitution on the training data. The page states that the cutoff prioritises manual review; it is not an approval decision.

4. **FP-Growth and patterns.** The mined itemsets, the two default rules, and their support, confidence, and lift. This page connects the model results to the simpler rule-based view.

The separation between training and serving matters. A dashboard that retrained on every page load would produce numbers that drift from the report and would make the review queue depend on the current random state. By freezing the artifacts, the console shows exactly the results discussed in this chapter.

The next chapter draws the results together, states the recommendation, and sets out the limitations and the work that would be needed before deployment.

Chapter 7

Results, Conclusions and Future Scope

This chapter brings the analysis together. It states which model was selected, what the model and the exploratory work say about default risk, what a credit team could reasonably do with the results, and what remains before the prototype could be used on live loans.

7.1 Summary of Results

The project built a loan-level feature table from the PKDD'99 data and compared six classifiers under a baseline and a SMOTE arm. The selected configuration is Gradient Boosting without resampling, tuned with a shallow ensemble of 300 stumps, a learning rate of 0.05, and 70% subsampling. Under repeated stratified five-fold cross-validation with ten repeats, it reaches a mean F1 of 0.571 with a standard deviation of 0.120, an average precision of 0.654, and a ROC-AUC of 0.864.

Random Forest is close enough to deserve equal mention. Its tuned baseline reaches an F1 of 0.556, an average precision of 0.661, and a ROC-AUC of 0.877. The F1 difference is 0.015 against a fold-to-fold spread of about 0.11, so the two models are statistically indistinguishable on that metric. Gradient Boosting was selected because F1 was the criterion fixed before the comparison; Random Forest is the better ranker and a credible alternative if the review queue is judged by average precision.

Table 7.1: The two strongest configurations under repeated cross-validation.

Model	F1	AP	AUC	Role
Gradient Boosting (baseline)	0.571 ± 0.120	0.654	0.864	Selected by the pre-specified F1 criterion.
Random Forest (baseline)	0.556 ± 0.112	0.661	0.877	Best ranking metrics; close alternative.

The held-out test table tells a consistent but noisier story. With default hyperparameters, Random Forest has the highest test F1 (0.519) and Gradient Boosting the highest average precision (0.583). The test set has only 19 defaults, so those differences are descriptive rather than decisive. The repeated cross-validation table is the stronger evidence because it averages 50 estimates.

7.2 Key Risk Signals

Three signals recur across the analysis.

1. **Balance capacity.** `min_balance` and `balance_at_loan` are the two most important features in the tuned Random Forest, at 0.261 and 0.116. Defaulted borrowers were more likely to have a low balance before the loan and to have touched a negative balance at some point. The exploratory medians point in the same direction: a balance at the loan date of about 22,000 for defaults against 39,500 for good loans.

2. **Loan burden relative to balance.** Loan amount and monthly payment are positively associated with default, and the combination matters more than either variable alone. Loans in the low-balance, high-amount quadrant default at 23.3%, compared with 4.9% in the high-balance, low-amount quadrant.

3. **Activity is secondary.** Transaction frequency and volume carry some signal, but they do not separate the classes cleanly. The busier segment defaults at 9.2% and the quieter segment at 12.4%, a modest gap. Activity is useful context, not a primary screen.

Age and gender are close to noise in this dataset. Their correlations with the target are 0.01 and 0.02, their importance scores are 0.025 and 0.005, and the age distribution shows no high-risk band. The report does not recommend using either variable in a lending rule.

The association rule mining adds one sharp but narrow finding: all 27 accounts that ever had a negative pre-loan balance defaulted. That is a 100% confidence rule with a lift of 8.97, covering 35.5% of defaults. It is based on 27 cases, so it should be treated as a hypothesis for a larger sample, not as a policy. The second rule, cash withdrawal activity, has a lift of only 1.17 and is not actionable.

7.3 Innovation Added to the Classification Algorithm

The innovation is a hybrid classification framework: a supervised classifier ranks loans, association rules provide a separate high-precision audit layer, and customer segments provide portfolio context. The estimator itself is standard. The contribution is in how the classifier is represented, validated, and combined with interpretable rules, where four choices distinguish the project from a standard train-and-report exercise.

1. **Pre-loan behavioural representation.** The 13 behavioural features are computed only from transactions dated strictly before the loan grant date. The representation includes the balance at the loan date, the minimum balance over the observed history, and the length of that history. This turns a raw transaction log into a decision-time profile and closes the most common leakage path in account-level credit models.

2. **Loan burden relative to balance.** Loan amount and monthly payment are read against the pre-loan balance. The resulting quadrant separates a 23.3% default rate in the low-balance, high-amount cell from 4.9% in the opposite cell. The interaction is both a model input and a transparent explanation for a reviewer.

3. **Two-arm imbalance validation.** Every classifier is trained on the natural distribution and on a SMOTE-balanced copy, then compared with 50 repeated cross-validation estimates. The result is negative for the common assumption that resampling always helps: SMOTE lowers F1 for five of the six tuned configurations. Testing that assumption, rather than adopting it by default, is part of the contribution.

4. **Rule-plus-model review layer.** The classifier ranks the portfolio, while association rule mining provides a separate high-precision audit signal. All 27 accounts with a negative pre-loan balance defaulted, a rule with 100% confidence and lift 8.97. The rule is shown beside the model score rather than blended into it, so a reviewer can see a simple reason for a flag. KMeans segments add portfolio context without using the target.

The result is a reproducible, leakage-controlled pipeline that combines a classifier with interpretable rules and a review interface. The innovation is a workflow for this dataset, not a claim of algorithmic novelty.

7.4 Business Recommendations

The results support a review-oriented workflow rather than an automated decision.

1. **Rank the queue; do not automate the decision.** The model is good at putting likely defaults near the top of a list, but its precision at useful recall is far from certain. A human reviewer should see the score, the balance history, and the loan terms together.

2. **Use a balance-capacity check as an interpretable first pass.** A simple rule that flags loans with `balance_at_loan` below 20,000 identifies 97 of 682 loans (14.2%), catches 34 of 76 defaults (44.7%), and has a precision of 35.1% against an 11.1% base rate. It is not a substitute for the model, but it is a useful audit check and an explanation for reviewers who want a transparent reason for the flag.

3. **Prioritise the low-balance, high-amount quadrant.** The 23.3% default rate in that cell is more than twice the portfolio average. A reviewer working through a long queue should start there, then use the model score to order the rest.

4. **Do not build rules on age or gender.** The data does not support them as risk drivers, and using them would create fairness and regulatory exposure for no measurable gain.

5. **Do not treat SMOTE as a default remedy.** It helped KNN and raised recall in some arms, but it lowered F1 for five of the six tuned configurations. A cost-sensitive threshold or class weights are better first experiments for this dataset.

6. **Calibrate before setting a cost-based cutoff.** The current scores rank loans but are not calibrated probabilities. If the bank wants to compare the expected cost of a default with the expected profit of a good loan, it needs a calibration step and a threshold chosen from those costs.

7. **Revalidate on recent data and monitor drift.** The dataset covers 1993 to 1998. Borrower behaviour, interest rates, and data collection have changed. A model trained on this period should be treated as a methodological prototype, not as a production scorecard.

7.5 Limitations

The limitations fall into four groups.

Sample size. There are only 76 defaults. The test partition contains 19. A single split can move recall by more than five percentage points when one loan changes side, and the cross-validation standard deviations are around 0.10 to 0.13. The reported means are estimates with wide uncertainty, not precise performance guarantees.

Evaluation optimism. The tuned cross-validation means reuse the same folds for hyperparameter selection and reporting. This makes them mildly optimistic. A nested cross-validation,

where the tuning is repeated inside each outer fold, would give a cleaner estimate of the full modelling procedure.

Data coverage. The features describe account transactions, loan terms, age, and gender. They do not include income, employment, collateral, or credit bureau history, which are central to real credit assessment. The transaction history is left-censored for early loans, and the dataset is from a single Czech bank in the 1990s. District was retained for profiling but excluded from the model matrix, so regional effects are not modelled.

Interpretation. Feature importance from a tree ensemble is not a causal effect. Correlated features share credit, and the near-duplicate cash-flow pair makes individual rankings unstable. The negative-balance rule is based on 27 accounts and has a wide confidence interval. The Streamlit review queue uses out-of-fold scores for retrospective analysis; it is not a live scoring service.

7.6 Future Scope

The natural next steps follow from the limitations.

- **Nested cross-validation and repeated holdout.** Estimate the tuning procedure itself, not only the tuned model, and report uncertainty intervals around the selected model's F1 and AP.
- **Cost-sensitive learning.** Replace the symmetric F1 objective with a cost matrix that reflects the actual loss from a missed default and the cost of reviewing a good customer. This would connect the model to the bank's economics.
- **Probability calibration.** Fit Platt scaling or isotonic regression on a held-out fold, check the calibration curve, and only then use probability thresholds in a decision rule.
- **Model explanation.** Add SHAP values or permutation importance to show which features drive an individual score. This would help reviewers trust or challenge a flag.
- **Richer data.** Add income, employment, collateral, and credit bureau records if they are available. Even within the PKDD'99 schema, the transaction categories and counterparty fields could support more detailed behavioural features.

- **Temporal validation.** Train on earlier loans and test on later ones to measure how the model degrades as borrower behaviour changes. This is a better proxy for deployment than a random split.
- **Deployment and monitoring.** Put the feature pipeline and model behind a versioned service, log inputs and decisions, monitor score distributions and default rates, and define a retraining schedule. The current Streamlit console is a presentation layer, not a production system.
- **Fairness and governance.** Audit performance across protected groups, document the intended use, and define the human review and appeal process before any live use.

7.7 Conclusion

The project asked whether pre-loan account behaviour can improve the identification of loans that will run into difficulty. The answer is a qualified yes. A compact set of 18 features, dominated by balance history and loan burden, supports a Gradient Boosting model with a cross-validated F1 of 0.571 and an average precision of 0.654, roughly six times the 11.14% base rate. The same features produce simple, interpretable signals: a low balance at the loan date, a large loan relative to that balance, and a history that has touched a negative balance.

The qualified part matters. The default class has only 76 examples, the test set has 19, and the top two models are separated by less than the fold-to-fold noise. SMOTE did not deliver the improvement that is often assumed, and the demographic variables contributed almost nothing. The honest conclusion is that the model is a useful ranking and review tool on this dataset, not an autonomous credit decision system. Its value lies as much in the discipline of the pipeline, the leakage controls, and the transparent evaluation as in the final score. Those practices, rather than any single accuracy figure, are what the project contributes to a real credit-risk workflow.

References

R1 Dataset

[1] P. Berka, "PKDD'99 Discovery Challenge: Financial Dataset," 1999. [Online]. Available: `https://www.kaggle.com/datasets/siavashraz/1999-czech-financial-dataset`

R2 Libraries and Tools

[2] F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. [Online]. Available: `https://scikit-learn.org`

[3] G. Lemaître, F. Nogueira, and C. K. Aridas, "Imbalanced-learn: A Python Toolbox to Tackle the Curse of Imbalanced Datasets in Machine Learning," *Journal of Machine Learning Research*, vol. 18, no. 17, pp. 1–5, 2017. [Online]. Available: `https://imbalanced-learn.org`

[4] W. McKinney, "Data Structures for Statistical Computing in Python," in *Proceedings of the 9th Python in Science Conference*, 2010, pp. 56–61. [Online]. Available: `https://pandas.pydata.org`

[5] J. D. Hunter, "Matplotlib: A 2D Graphics Environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007. [Online]. Available: `https://matplotlib.org`

[6] M. Waskom, "Seaborn: Statistical Data Visualization," *Journal of Open Source Software*, vol. 6, no. 60, p. 3021, 2021. [Online]. Available: `https://seaborn.pydata.org`

[7] T. Kluyver *et al.*, "Jupyter Notebooks — a Publishing Format for Reproducible Computational Workflows," in *Positioning and Power in Academic Publishing*, IOS Press, 2016, pp. 87–90. [Online]. Available: `https://jupyter.org`

R3 Methodological References

[8] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic Minority Over-sampling Technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.

[9] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[10] J. H. Friedman, "Greedy Function Approximation: A Gradient Boosting Machine," *Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.

[11] R. Wirth and J. Hipp, "CRISP-DM: Towards a Standard Process Model for Data Mining," in *Proceedings of the 4th International Conference on the Practical Application of Knowledge Discovery and Data Mining*, 2000, pp. 29–39.